

Hands-On Lab | Practical 2

Inheritance and Polymorphism

 USTH
  Object-Oriented Programming
  3 hours
  JDK 21/25 + IntelliJ IDEA
 MovieNight@USTH

MovieNight@USTH grows: besides films the club now screens **series marathons**, **short films** and **documentaries**, and some members are VIPs. All these kinds of titles must live in one catalogue and be treated the same way - this is the practical where Practical 1's classes get parents and children. Continue in your `movienight` project; **each exercise is done in a different way** - the badge under its title tells you how.

Before you start

Everything from Lectures 1 to 6 is allowed: constructors, `this`, `static`, `extends`, `super(...)`, `super.method()`, `@Override`, `protected`, polymorphic arrays, and `toString()` / `equals()` inherited from `Object`. **Not yet:** abstract classes and interfaces (Lectures 7-8), exceptions (Lecture 9) - so every class in this practical can be instantiated, and a refused value is reported with a message. Write `@Override` above every overriding method.

Exercise 1 Title

 **Refactor** Two classes that repeat each other. Pull the common part up into a parent without changing the output.

The club's catalogue started with two unrelated classes. Read them side by side:

```
public class Film {
    private String name;
    private int year;
    private int duration;

    public Film(String name, int year,
                int duration) {
        this.name = name;
        this.year = year;
        this.duration = duration;
    }
    public String getName() { return name; }
    public int getYear() { return year; }
    public int getDuration() { return duration; }

    public void printInfo() {
        System.out.println(name + " (" + year
            + ") - " + duration + " min");
    }
}
```

```
public class Series {
    private String name;
    private int year;
    private int episodes;
    private int minutesPerEpisode;

    public Series(String name, int year,
                  int episodes, int minutesPerEpisode) {
        this.name = name;
        this.year = year;
        this.episodes = episodes;
        this.minutesPerEpisode = minutesPerEpisode;
    }
    public String getName() { return name; }
    public int getYear() { return year; }
    public int getEpisodes() { return episodes; }
    public int getMinutesPerEpisode() {
        return minutesPerEpisode;
    }
    public void printInfo() {
        System.out.println(name + " (" + year + ") - "
            + episodes * minutesPerEpisode + " min");
    }
}
```

Write a Java class `Title` holding everything the two classes share, make `Film` and `Series` **extend** it, and remove every duplicated line:

- `name` and `year` exist **only** in `Title`; the subclass constructors call `super(name, year)`
- `printInfo()` is written **once**, in `Title`; it gets the length from a method `runningTime()` that `Film` and `Series` **override** (`Title` itself returns 0)
- The tester `TitleTestDrive` - a Film *Dune* (2021, 155 min) and a Series *Bluey* (2018, 9 episodes of 7 min), `printInfo()` on both



Expected output

```

Dune (2021) - 155 min
Bluey (2018) - 63 min
  
```

Exercise 2 Programme

 **Tests first** The tester is written and uses one array for everything. Make it print the expected output.

The tester `ProgrammeTestDrive` is given and **must not be modified**. Complete `Title`, `Film`, `Series` and write the new class `ShortFilm` so that it compiles and prints the expected output, line for line.

```
public class ProgrammeTestDrive {
    public static void main(String[] args) {
        Title[] programme = new Title[3];
        programme[0] = new Film("Dune", 2021, 155);
        programme[1] = new Series("Bluey", 2018, 9, 7);
        programme[2] = new ShortFilm("The Neighbors' Window", 2019,
            20);

        int total = 0;
        for (Title t : programme) {
            System.out.println(t);
            total = total + t.runningTime();
        }
        System.out.println("Total: " + total + " min");

        Title longest = programme[0];
        for (Title t : programme) {
            if (t.runningTime() > longest.runningTime()) {
                longest = t;
            }
        }
        System.out.println("Longest: " + longest.getName());
    }
}
```

Title

name
year
runningTime()
kind()
toString()
getters

Expected output

```
Film: Dune (2021) - 155 min
Series: Bluey (2018) - 63 min (9 x 7 min)
Short film: The Neighbors' Window (2019) - 20 min
Total: 238 min
Longest: Dune
```

- `System.out.println(t)` prints `t.toString()` : write `toString()` **once**, in `Title`, from `kind()` and `runningTime()` ; `Film` and `ShortFilm` override `kind()` only ("Film", "Short film")
- `ShortFilm` extends `Film` - it inherits duration and running time, a 3-level chain `Title` → `Film` → `ShortFilm`
- `Series` adds (9 x 7 min) to the parent's text with `super.toString()`

Exercise 3 Catalogue

 **Make it compile** Five compile errors, one at a time. Read what javac says, fix, compile again.

A member wrote the catalogue in one file. It does **not compile**: `javac` reports **errors**, which you should find and fix it. Type the file, compile it, and fix the errors one by one - recompile after each fix and watch the list shrink. When the compiler is finally happy, compare the output with the expected one: **one line will still be wrong**, and no error message will tell you why.

```

1  class Title {
2      private String name;
3      private int year;
4
5      Title(String name, int year) {
6          this.name = name;
7          this.year = year;
8      }
9      public String getName() { return name; }
10     public int runningTime() { return 0; }
11     public String kind() { return "Title"; }
12 }
13
14 class Film extends Title {
15     private int duration;
16
17     Film(String name, int year, int duration) {
18         this.duration = duration;
19         super(name, year);
20     }
21     @Override
22     public int runningTime() { return duration; }
23     @Override
24     private String kind() { return "Film"; }
25 }
26
27 class Series extends Title {
28     private int episodes;
29     private int minutesPerEpisode;
30
31     Series(String name, int year, int episodes, int
32           minutesPerEpisode) {
33         this.episodes = episodes;
34         this.minutesPerEpisode = minutesPerEpisode;
35     }
36     @Override
37     public int runningTime() { return episodes *
38           minutesPerEpisode; }
39     public int getEpisodes() { return episodes; }
40 }
41
42 public class Catalogue {
43     public static void main(String[] args) {
44         Title[] items = { new Film("Dune", 2021, 155),
45                           new Series("Bluey", 2018, 9, 7) };
46         for (Title t : items) {
47             System.out.println(t.kind() + ": " + t.getName()
48                               + " - " + t.runningTime() + " min");
49         }
50         System.out.println(items[1].getEpisodes() + " episodes");
51     }
52 }

```

Expected output

```

Film: Dune - 155 min
Series: Bluey - 63 min
9 episodes

```

Rules

- Do not delete a method or an `@Override` to silence an error - fix the cause
- Line 48 must be fixed **without** changing class `Series`: think about which remote you are holding
- Keep a note of the **error messages** - you will meet them again

Exercise 4 Documentary

Extend it The tester already uses a class that does not exist. Write that class - and nothing else.

The club adds documentaries. Somebody has already updated the tester below; it does not compile yet because the class `Documentary` is missing. Write `Documentary.java` so that the tester compiles and prints the expected output. **Do not modify the tester, nor Title, Film, Series or ShortFilm.**

```
public class CatalogueTestDrive {
    public static void main(String[] args) {
        Title[] catalogue = {
            new Film("Dune", 2021, 155),
            new Series("Bluey", 2018, 9, 7),
            new Documentary("Free Solo", 2018, 100, "climbing"),
            new ShortFilm("The Neighbors' Window", 2019, 20)
        };
        for (Title t : catalogue) {
            System.out.println(t);
        }
        int total = 0;
        for (Title t : catalogue) {
            total = total + t.runningTime();
        }
        System.out.println(catalogue.length + " titles, "
            + total + " min in total");
    }
}
```

Documentary

topic
kind()
toString()
getTopic()

Expected output

```
Film: Dune (2021) - 155 min
Series: Bluey (2018) - 63 min (9 x 7 min)
Documentary: Free Solo (2018) - 100 min [climbing]
Short film: The Neighbors' Window (2019) - 20 min
4 titles, 338 min in total
```

- Choose the parent yourself: a documentary is a kind of film with a `topic` - the fourth constructor argument
- `toString()` adds `[climbing]` to the parent's text; the total of the running times must come out right without any change to the loops

Exercise 5 Member and VipMember

Break the code Write the two classes, then write a tester that attacks them. They must survive.

Members pay for tickets; VIP members get a discount. Write a Java class `Member`, a Java class `VipMember` that **extends** it, and a tester `MemberTestDrive`.

- Apply the "Encapsulation" concept; constructors with `super(...)`
- `price(double basePrice)` returns what this member pays: the base price for a `Member`, the base price reduced by `discountPercent` for a `VipMember`
- Override `toString()` in both classes; `VipMember` reuses the parent's text with `super.toString()` and adds `- VIP 20%`
- Override `equals(Object o)` in `Member`: two members are equal when they have the **same student ID**, whatever their names or classes
- The tester prints the two members, the prices they pay for a 5.0-credit ticket, and compares `linh` with a second object having the same ID using both `==` and `equals`

Member

name
studentId
price(double)
toString()
equals(Object)
getters



VipMember

discountPercent
price(double)
toString()
getter, setter

Then break it. Your tester must also contain these three attacks, and your classes must survive all of them with a sensible result instead of a crash: (a) `linh.equals(null)`; (b) `linh.equals("BI13-123")` - a String is not a Member; (c) a VIP discount of 150%, which would make the price negative.

Hint

Inside `equals`, the parameter is an `Object`. Before comparing student IDs, check what it really is - `o instanceof Member` - and only then convert it: `Member other = (Member) o;`

Exercise 6 Screening

Q Code review The code compiles and runs. Label every problem before you fix anything.

A member decided that “a screening is a film with a room and a date” and wrote the class below. It compiles and runs, but almost every line has a problem. Review it first: for every problem, write a comment in the code with one of the labels **wrong IS-A** (the inheritance relationship itself is wrong), **shadowing** (a field of the parent is declared again), **encapsulation** (attributes are not hidden), **fake override** (looks like an override, is not one) or **broken behaviour** (an inherited method now gives a wrong answer). Then fix the class as much as possible - the diagram shows the intended design.

```

1 public class Screening extends Film {
2     public String room;
3     public String date;
4     protected String name;
5     public int sold;
6
7     public Screening(String name, int year, int duration,
8                     String room, String date) {
9         super(name, year, duration);
10        this.name = name;
11        this.room = room;
12        this.date = date;
13    }
14
15    public int runningTime() {
16        return 0;
17    }
18
19    public String toString(String prefix) {
20        return prefix + name + " @ " + room + ", " + date;
21    }
22
23    public static void main(String[] args) {
24        Screening s = new Screening("Dune", 2021, 155,
25                                   "2H-201", "2026-10-03 19:00");
26        s.sold = 12;
27        System.out.println(s.toString("Screening: "));
28        System.out.println(s);
29        System.out.println("Running time: " + s.runningTime() + "
30                           min");
31    }
32 }

```

Screening

- film: Film
- room: String
- date: String
- sold: int
- + sell(int n): void
- + toString(): String
- + getters

It prints today

```

Screening: Dune @ 2H-201, 2026-10-03
19:00
Film: Dune (2021) - 0 min
Running time: 0 min

```